



Trabajo Práctico Integrador

Programación 2

FOOD STORE

Sistema de gestión de pedidos de comida
Aplicación Java de consola con POO y colecciones

Estudiante	Santino Arias
Materia	Programación 2
Proyecto	Food Store - App de consola
Lenguaje	Java 21
IDE utilizado	NetBeans
Fecha	15 Junio de 2026

Resumen

El proyecto implementa una aplicación de consola para administrar categorías, productos, usuarios y pedidos. La solución se organizó en capas simples, utiliza colecciones en memoria, aplica baja lógica, validaciones de entrada, excepciones propias y una interfaz para calcular totales de pedido.

Indice

1. Introducción	3
2. Objetivos del proyecto.....	3
3. Alcance funcional	3
4. Tecnologías utilizadas	4
5. Arquitectura y organización del código	4
5.1 Responsabilidad de cada capa	5
6. Modelo de dominio.....	5
6.1 Relación 1:1 Pedido - ComprobantePedido	5
7. Implementación de POO	6
7.1 Encapsulamiento.....	6
7.2 Herencia	6
7.3 Polimorfismo y sobrescritura	6
7.4 Interfaces	6
8. Reglas de negocio implementadas.....	7
9. Manejo de excepciones	7
10. Flujo de uso de la aplicación	7
11. Ejecución desde NetBeans	8
12. Pruebas realizadas	8
13. Decisiones técnicas destacadas	10
14. Dificultades y soluciones	10
15. Repositorio y entrega	10
16. Conclusión	10
17. Bibliografía y recursos consultados.....	10

1. Introducción

El presente informe documenta el desarrollo del Trabajo Práctico Integrador de Programación 2, denominado Food Store. El sistema fue realizado como una aplicación de consola en Java, orientada a la gestión básica de un negocio de comidas. Su finalidad principal es permitir la administración de categorías, productos, usuarios y pedidos, aplicando los contenidos trabajados durante la materia: programación orientada a objetos, herencia, encapsulamiento, colecciones, interfaces, excepciones y separación de responsabilidades.

La solución se diseñó con un enfoque simple y mantenible. No se utilizó base de datos, ya que el alcance de la entrega establece almacenamiento en memoria durante la ejecución del programa. Por este motivo, los objetos se guardan en colecciones dinámicas y se manipulan a través de servicios que concentran la lógica de negocio.

2. Objetivos del proyecto

- Modelar correctamente las entidades principales del dominio Food Store.
- Aplicar herencia mediante una clase abstracta Base compartida por las entidades.
- Implementar operaciones CRUD completas en consola para categorías, productos, usuarios y pedidos.
- Utilizar colecciones en memoria para almacenar y buscar objetos.
- Aplicar baja lógica mediante el atributo eliminado, evitando borrar físicamente los datos.
- Validar entradas del usuario para evitar cierres inesperados del programa.
- Crear y capturar excepciones propias para representar errores de negocio.
- Incorporar la interfaz Calculable para el cálculo del total de pedidos.
- Separar el código en paquetes según responsabilidad: entidades, servicios, interfaz de consola, enums y excepciones.

3. Alcance funcional

El sistema permite operar desde un menú principal de consola. Cada módulo abre un submenú con sus acciones correspondientes. El usuario operador puede realizar altas, consultas, modificaciones y eliminaciones lógicas sin necesidad de iniciar sesión.

Módulo	Operaciones implementadas	Observaciones
Categorías	Listar, crear, editar y eliminar lógicamente.	No se permite duplicar nombres y se controla la existencia de productos activos.
Productos	Listar general, listar por categoría, crear, editar y eliminar lógicamente.	Se valida precio, stock, categoría asociada y disponibilidad.
Usuarios	Listar, crear, editar y eliminar lógicamente.	El mail se valida como único para evitar registros repetidos.
Pedidos	Listar, crear con detalles, actualizar estado/forma de pago y eliminar lógicamente.	Calcula subtotales y total; descuenta stock al crear el pedido.

4. Tecnologías utilizadas

Tecnología	Uso dentro del proyecto	Motivo de elección
Java 21	Lenguaje principal de desarrollo.	Permite aplicar POO, colecciones, excepciones y enums de forma clara.
NetBeans	IDE utilizado para abrir, compilar y ejecutar el proyecto.	Entorno conocido, ordenado y compatible con proyectos Maven.
Maven	Estructura estándar del proyecto y configuración de compilación.	Facilita reproducibilidad y ejecución en otros equipos.
Colecciones Java	ArrayList para almacenar entidades en memoria.	Permiten altas, bajas lógicas, búsquedas y listados durante la ejecución.
GitHub	Repositorio público de entrega.	Permite compartir el código y documentar el proyecto con README.

5. Arquitectura y organización del código

El proyecto se organizó por paquetes para mantener una separación clara entre el modelo de dominio, la lógica de negocio y la interacción con el usuario. Esta decisión evita concentrar toda la solución en el Main y facilita la lectura, prueba y mantenimiento del código.

```
src/main/java/integrado/prog2/
├── Main.java
├── config/
│   └── DatosIniciales.java
├── entities/
│   ├── Base.java
│   ├── Categoria.java
│   ├── Producto.java
│   ├── Usuario.java
│   ├── Pedido.java
│   ├── DetallePedido.java
│   └── ComprobantePedido.java
├── enums/
│   ├── Rol.java
│   ├── Estado.java
│   └── FormaPago.java
├── exceptions/
│   ├── NegocioException.java
│   ├── EntidadNoEncontradaException.java
│   ├── ValidacionException.java
│   └── StockInsuficienteException.java
├── interfaces/
│   └── Calculable.java
├── services/
│   ├── CategoriaService.java
│   ├── ProductoService.java
│   ├── UsuarioService.java
│   └── PedidoService.java
└── ui/
    ├── AppMenu.java
    └── InputHelper.java
```

5.1 Responsabilidad de cada capa

Capa / paquete	Responsabilidad principal
entities	Define las clases del dominio y sus relaciones.
services	Contiene reglas de negocio, validaciones, búsquedas y operaciones CRUD.
ui	Gestiona el menú de consola, lectura de datos y mensajes al usuario.
exceptions	Agrupar excepciones propias para errores de validación, stock o entidades inexistentes.
enums	Define valores cerrados para rol, estado y forma de pago.
interfaces	Declara comportamientos comunes, como el cálculo de totales.
config	Carga datos iniciales para facilitar pruebas y demostración.

6. Modelo de dominio

El modelo se basa en una clase abstracta Base que centraliza atributos comunes. A partir de ella heredan las entidades principales del sistema. Esta decisión reduce duplicación y mantiene una estructura uniforme para el manejo de identificadores, fechas de creación y baja lógica.

Clase	Hereda de	Atributos relevantes	Relaciones principales
Base	-	id, eliminado, createdAt	Clase abstracta común.
Categoria	Base	nombre, descripcion	1:N con Producto.
Producto	Base	nombre, precio, stock, imagen, disponible	N:1 con Categoria; usado por DetallePedido.
Usuario	Base	nombre, apellido, mail, celular, contraseña, rol	1:N con Pedido.
Pedido	Base	fecha, estado, total, formaPago	1:N con DetallePedido; N:1 con Usuario; 1:1 con ComprobantePedido.
DetallePedido	Base	cantidad, subtotal	N:1 con Producto.
ComprobantePedido	Base	codigo, fechaEmision, importe	1:1 con Pedido.

6.1 Relación 1:1 Pedido - ComprobantePedido

Como extensión del modelo, se incorporó la clase ComprobantePedido para representar una relación 1:1 real y unidireccional desde Pedido hacia ComprobantePedido. Al crear un pedido correctamente, el sistema genera un comprobante asociado con un código, una fecha de emisión y el importe total.

Decisión técnica

La relación Pedido -> ComprobantePedido permite demostrar una asociación uno a uno efectiva en el código, manteniendo el sentido del dominio: cada pedido confirmado durante la creación posee un comprobante propio.

7. Implementación de POO

7.1 Encapsulamiento

Los atributos de las entidades se declararon como privados y el acceso se realiza mediante métodos públicos. Esto permite controlar cómo se modifican los datos y evita exponer directamente el estado interno de los objetos.

7.2 Herencia

Todas las entidades principales heredan de Base. De esta forma, comparten id, eliminado y createdAt, además de métodos comunes como isActive(), eliminar(), equals() y hashCode().

7.3 Polimorfismo y sobrescritura

Cada entidad implementa su propio método toString() para mostrar información útil en los listados de consola. Además, Pedido implementa la interfaz Calculable, lo que permite tratar el cálculo de total como un comportamiento definido por contrato.

7.4 Interfaces

La interfaz Calculable declara el método calcularTotal(). La clase Pedido lo implementa recorriendo sus detalles y sumando subtotales. Esta decisión separa el comportamiento de cálculo del resto de las operaciones del menú.

```
public interface Calculable {
    void calcularTotal();
}

public class Pedido extends Base implements Calculable {
    @Override
    public void calcularTotal() {
        total = detalles.stream()
            .filter(DetallePedido::isActive)
            .mapToDouble(DetallePedido::getSubtotal)
            .sum();
    }
}
```

8. Reglas de negocio implementadas

Regla	Dónde se controla	Resultado esperado
Nombre de categoría no vacío y único.	CategoriaService	Evita categorías duplicadas o sin nombre.
Precio y stock de producto no negativos.	ProductoService	Evita datos inválidos en el catálogo.
Producto asociado a categoría existente y activa.	ProductoService	Mantiene coherencia entre producto y categoría.
Mail de usuario único.	UsuarioService	Evita usuarios repetidos por correo.
Pedido con usuario existente y activo.	PedidoService	No permite pedidos sin responsable.
Detalle con cantidad mayor a cero.	Pedido / PedidoService	Evita subtotales inválidos.
Stock suficiente al crear pedido.	PedidoService	Evita vender más unidades de las disponibles.
Baja lógica en todas las entidades principales.	Servicios CRUD	Conserva historial y evita borrar físicamente.

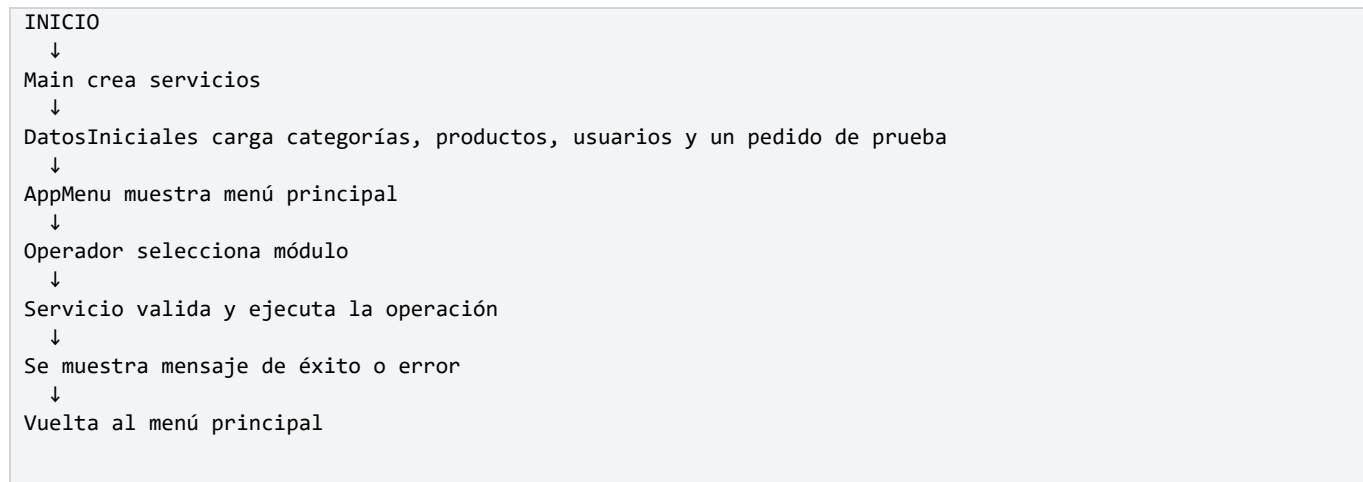
9. Manejo de excepciones

El proyecto utiliza excepciones propias para representar errores esperables del dominio. Estas excepciones permiten informar el problema sin finalizar la ejecución del programa. En el menú, las operaciones se encuentran protegidas con bloques try-catch para mostrar mensajes claros y retornar al flujo normal.

Excepción	Uso principal
NegocioException	Clase base para errores propios del sistema.
ValidacionException	Datos incompletos, negativos o inválidos.
EntidadNoEncontradaException	IDs inexistentes o entidades eliminadas.
StockInsuficienteException	Intento de crear pedido sin stock suficiente.

10. Flujo de uso de la aplicación

Al ejecutar el proyecto, la clase Main crea los servicios, carga datos iniciales y luego inicia AppMenu. El usuario navega por menús y submenús. Cada acción solicita datos, valida la entrada y delega la operación al servicio correspondiente.



11. Ejecución desde NetBeans

1. Abrir NetBeans.
2. Seleccionar File > Open Project.
3. Elegir la carpeta raíz del proyecto FoodStoreTPI.
4. Verificar que el proyecto reconozca Java 21.
5. Ejecutar Clean and Build para compilar.
6. Ejecutar Run Project. Si NetBeans solicita clase principal, seleccionar integrado.prog2.Main.

CAPTURA 1 - Proyecto abierto en NetBeans

```
cd C:\Users\arias\OneDrive\NOTEBOK\Documentos\UTN\SEGUNDO CUATRIMESTRE\Programacion II\INTEGRADOR\FoodStoreTPI\FoodStoreTPI; "JAVA_HOME=C:\Program Files\Java\jdk-21" cmd /c "%CD%\Users\arias\Downloads\netbeans-29-bin\netbeans
Scanning for projects...

-----< integrado.prog2:FoodStoreTPI >-----
Building FoodStoreTPI 1.0.0
from pom.xml
-----[ jar ]-----

--- resources:3.3.1:resources (default-resources) @ FoodStoreTPI ---
skip non existing resourceDirectory C:\Users\arias\OneDrive\NOTEBOK\Documentos\UTN\SEGUNDO CUATRIMESTRE\Programacion II\INTEGRADOR\FoodStoreTPI\FoodStoreTPI\src\main\resources

--- compiler:3.13.0:compile (default-compile) @ FoodStoreTPI ---
Nothing to compile - all classes are up to date.

--- exec:3.5.1:exec (default-cli) @ FoodStoreTPI ---
```

12. Pruebas realizadas

Para verificar el funcionamiento se realizaron pruebas manuales desde consola. Se recorrieron los CRUD principales y también casos de error para comprobar que el sistema no se cierre ante entradas incorrectas.

Caso de prueba	Entrada / acción	Resultado esperado	Estado
Listar categorías	Menú Categorías > Listar	Muestra categorías activas precargadas.	Correcto
Crear categoría	Nombre y descripción válidos	Genera ID y aparece en listado.	Correcto
Duplicar categoría	Mismo nombre existente	Informa error de validación.	Correcto
Crear producto	Datos válidos y categoría activa	Producto creado con ID.	Correcto
Producto con precio negativo	Precio menor a cero	Informa error y no crea producto.	Correcto
Crear usuario	Mail nuevo	Usuario creado correctamente.	Correcto
Duplicar mail	Mail ya existente	Informa error y cancela operación.	Correcto
Crear pedido	Usuario + productos + cantidades	Crea pedido, detalles, total y comprobante.	Correcto
Stock insuficiente	Cantidad superior al stock	Cancela creación e informa error.	Correcto
Baja lógica	Eliminar entidad con confirmación	Marca eliminado=true y no aparece en listados.	Correcto

CAPTURA 2 - Menú principal funcionando en consola

```

=====
SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:

```

CAPTURA 3 - Creación de pedido con detalles y total calculado

```

CREAR PEDIDO
=====
ID: 1 | Usuario: Santino Arias | Mail: santino@foodstore.com | Celular: 2990000000 | Rol: ADMIN
ID: 2 | Usuario: Juan Perez | Mail: juan.perez@mail.com | Celular: 2991111111 | Rol: USUARIO
ID de usuario: 1
Forma de pago
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione: 2

Productos disponibles:
ID: 1 | Producto: Hamburguesa Clasica | Precio: $6500,00 | Stock: 19 | Disponible: Si | Categoria: Hamburguesas
ID: 2 | Producto: Hamburguesa Doble | Precio: $8200,00 | Stock: 14 | Disponible: Si | Categoria: Hamburguesas
ID: 3 | Producto: Pizza Muzza | Precio: $9000,00 | Stock: 12 | Disponible: Si | Categoria: Pizzas
ID: 4 | Producto: Pizza Napolitana | Precio: $9800,00 | Stock: 10 | Disponible: Si | Categoria: Pizzas
ID: 5 | Producto: Gaseosa 500ml | Precio: $1800,00 | Stock: 40 | Disponible: Si | Categoria: Bebidas
ID de producto: 2
Cantidad: 2
Agregar otro detalle (S/N): S

Productos disponibles:
ID: 1 | Producto: Hamburguesa Clasica | Precio: $6500,00 | Stock: 19 | Disponible: Si | Categoria: Hamburguesas
ID: 2 | Producto: Hamburguesa Doble | Precio: $8200,00 | Stock: 14 | Disponible: Si | Categoria: Hamburguesas
ID: 3 | Producto: Pizza Muzza | Precio: $9000,00 | Stock: 12 | Disponible: Si | Categoria: Pizzas
ID: 4 | Producto: Pizza Napolitana | Precio: $9800,00 | Stock: 10 | Disponible: Si | Categoria: Pizzas
ID: 5 | Producto: Gaseosa 500ml | Precio: $1800,00 | Stock: 40 | Disponible: Si | Categoria: Bebidas
ID de producto: 5
Cantidad: 2
Agregar otro detalle (S/N): n
Pedido creado correctamente. ID generado: 2
ID: 2 | Usuario: Santino Arias | Estado: PENDIENTE | Pago: TRANSFERENCIA | Total: $20000,00 | Fecha: 2026-06-09
Comprobante: CP-0002 | Fecha: 09/06/2026 10:25 | Importe: $20000,00

Presione ENTER para continuar...|

```

13. Decisiones técnicas destacadas

- Se utilizó almacenamiento en memoria con ArrayList para mantener el alcance de la consigna y simplificar las pruebas.
- Se separó la lógica de negocio del menú para evitar que Main concentre responsabilidades.
- Se implementó baja lógica en lugar de eliminación física, permitiendo conservar historial y relaciones.
- Se agregaron datos iniciales para que la demo sea más rápida y para validar listados desde el primer inicio.
- Se creó InputHelper para centralizar la lectura y validación de entradas en consola.
- Se agregó ComprobantePedido como relación uno a uno vinculada a Pedido, manteniendo sentido funcional dentro del dominio.

14. Dificultades y soluciones

Dificultad	Problema detectado	Solución aplicada
Validaciones de entrada	El usuario podía ingresar letras donde se esperaban números.	Se creó InputHelper con métodos de lectura robustos y repetición hasta entrada válida.
Baja lógica	Eliminar físicamente podía romper relaciones históricas.	Se usó el campo eliminado heredado de Base y filtros de activos en los listados.
Pedidos con detalles	Era necesario calcular subtotales, total y controlar stock.	La creación se centralizó en PedidoService y se usó Calculable para el total.
Unicidad	Podían repetirse mails o nombres de categorías.	Los servicios recorren colecciones antes de crear o editar.
Organización del proyecto	El menú podía quedar demasiado grande.	Se separaron servicios y entidades para dejar el menú enfocado en interacción.

15. Repositorio y entrega

El código fuente se entrega en un repositorio público de GitHub. El README incluye una descripción del sistema, instrucciones de ejecución, estructura del proyecto y enlaces de entrega.

Repositorio: <https://github.com/SantinoArias/TP-INTEGRADOR-PROGRAMACION2>

Video demostrativo: **[PEGAR LINK PÚBLICO DEL VIDEO]**

16. Conclusión

Food Store cumple con el objetivo de integrar los conceptos principales de Programación 2 en una aplicación concreta de consola. El sistema evidencia uso de clases, herencia, encapsulamiento, colecciones, interfaces, enums, excepciones propias, validaciones, baja lógica y separación de responsabilidades. Además, la estructura del proyecto permite comprender el flujo de ejecución y facilita futuras mejoras, como persistencia en base de datos, autenticación de usuarios o una interfaz gráfica.

17. Bibliografía y recursos consultados

- Material de cátedra: Consigna del Trabajo Práctico Integrador - Programación 2, 2026.
- Material de cátedra: Pautas de corrección del TPI - Programación 2, 2026.
- Documentación de Apache NetBeans: <https://netbeans.apache.org/>
- Documentación de Maven: <https://maven.apache.org/guides/>
- Documentación de GitHub: <https://docs.github.com/>
- OpenAI. ChatGPT, modelo GPT-5.5 Thinking.